

Design — how the isotope is carried

The conceptual foundation of `confluent-kafka-isotope`: what the isotope is, how it rides in Kafka headers, how produce- and consume-side edges are captured, and why the topology is modelled as a bipartite graph. For *running* the project see the [README](#) and the runbooks under [docs/](#).

Contents

- [The isotope and the bipartite model](#)
 - [Header layout](#)
 - [1. How the producer interceptor gets invoked](#)
 - [2. Kafka client interceptor vs explicit library calls](#)
 - [3. Why explicit calls for consume-side markers?](#)
 - [4. Why "bipartite"?](#)
-

The isotope and the bipartite model

An isotope is a lightweight tracing artifact attached to Kafka record headers. Like a biochemical isotope used to trace molecules through a metabolic pathway, it allows the journey of a record through an event-driven architecture to be observed and analyzed.

Like many software patterns, Isotope Tracing can be expressed in mathematical terms. This project renders a Kafka pipeline as a **bipartite graph** — services on one vertex set, topics on the other, with edges running both ways between them — so every produce edge, consume edge, and terminal consumer becomes a first-class citizen of a single topology view.

The tracer that makes this possible is the isotope carried in each record header and observed at two points. On the produce side, a Kafka producer interceptor stamps the isotope and appends one hop per `send()` call, creating the graph's produce edges in `hops[]`. On the consume side, `IsotopeContext.recordConsume(record, service, producer)` forwards the isotope to a value-less marker on `isotope_consume_edge_markers`, creating the consume edges. Services that consume and then produce call `IsotopeContext.adoptFromRecord(record)` in between so the trace identity survives each hop.

Apache Flink reads only the headers and reconstructs what the isotopes reveal: **end-to-end latency**, the **complete service→topic→service topology**, **drop and duplication rates**, and **pipeline coverage**.

Viewed through the biochemical analogy, Kafka records are the molecules moving through the system, while isotopes are the labels that make those movements observable. The goal is not to monitor the isotopes themselves, but to understand the behavior of the event-driven architecture they traverse. By following isotopes through topics and services, Apache Flink can reconstruct the pathways records take, measure how long they spend at each stage, detect where they are dropped or duplicated, and reveal the topology of the system as it actually operates.

A point worth affirming up front: **open-source Apache Flink and Confluent Cloud for Apache Flink (CCAF) run this the same way**. The identical isotope mechanism works against both **Confluent Platform for Apache Flink** (self-managed, the open-source Apache Flink runtime) and **CCAF** (managed)

— both driven here via Flink SQL plus an uploaded PTF JAR, no `DataStream` code on either side. Three design choices keep it that way: tagging happens in a Kafka **producer interceptor** (the one extension point both runtimes share via the broker), the on-wire **header** format is **JSON** (so Flink SQL can read the scalar fields with `CAST(headers [...] AS STRING)` and no UDF), and the two stateful reports (`LatencyPercentilesPTF`, `StuckTracePTF`) ship as a single JAR that registers identically on either runtime. Both are `ProcessTableFunctions` — even percentiles, which would naturally be an aggregate function, because CCAF does not support user-defined aggregates currently.

One asymmetry the runtimes don't share: **Flink-native SR-Protobuf**. CCAF supports SR-framed Protobuf as a Flink sink format via its topic catalog; Apache Flink open-source (the CP Flink runtime) ships `avro-confluent` but no SR-Protobuf counterpart. So Flink *report sinks* land on **Avro+SR on CP** and can be **Protobuf+SR on CCAF** — a runtime constraint, not a project preference. The demo *event* topics (next section) are unaffected because they're written by the Kafka producer client, not by Flink.

Header layout

Message **values** on the demo topics are **SR-framed Protobuf** (`ai.signalroom.kafka.isotope.proto.DemoEvent`) — the standard Confluent value format. The interceptors and reports are agnostic to value format because the isotope rides in headers; the Protobuf choice just gives the integration tests and the demo CLI a typed payload to work with. The **headers** are where the isotope lives, and they have two parts:

- **Header `x-isotope`** (JSON bytes) carries the full hop history, forwarded by every hop:
 - `t` — 16-byte **UUIDv7** trace ID ([RFC 9562](#)): 48-bit ms timestamp in the high bits + 74 bits random. Stable for the life of the trace, and lexicographic byte order matches creation order — sort trace IDs and you get chronological order for free. The random bits come from `ThreadLocalRandom`, not `SecureRandom`: a trace ID is a public observability identifier carried in Kafka headers, so the requirement is collision avoidance, not unpredictability — and `ThreadLocalRandom` delivers that without `SecureRandom`'s per-call cost on every produced record.
 - `o` — origin timestamp (ms) — same value as the timestamp embedded in the UUIDv7 trace ID; kept as its own field for typed access from Flink SQL without needing to decode the UUID bytes.
 - `s` — origin service name (set once, never reassigned)
 - `p` — origin pipeline name (e.g. `orders` vs `location`); like `s`, stamped once at the origin and forwarded unchanged on every hop, so reports can slice traces by which logical pipeline they belong to
 - `h` — ordered list of hops, each `{s: service, t: topic, m: tsMs}`
 - `x` — `true` if the hop list exceeded `MAX_HOPS = 32` and the oldest hop was evicted
- **Seven scalar headers** (UTF-8 strings) carry the most-recent-hop view so Flink SQL can read them via `CAST(headers['x-isotope-...'] AS STRING)` without parsing the JSON array (no UDF required on either CCAF or CP Flink). See [scripts/flink/README.md](#) for the full header table.

Example of the Isotope's two parts (formatted for readability; the actual header is in JSON bytes):

```
{
  "x-isotope-hop-count": "1",
  "x-isotope": "
```

```
{
  "t": "AZ690S80eG+9zufGfF2sbw==",
  "o": "1781291101966",
  "s": "order-intake-service",
  "p": "order",
  "h": [{"s": "order-intake-service",
    "t": "orders.placed",
    "m": "1781291101966"}],
  "x": false,
  "x-isotope-trace-id": "019ebd392f0e786fbdcee7c67c5dac6f",
  "x-isotope-this-service": "order-intake-service",
  "x-isotope-origin-service": "order-intake-service",
  "x-isotope-this-topic": "orders.placed",
  "x-isotope-origin-ts": "1781291101966",
  "x-isotope-pipeline": "order"
}
```

A producer with the isotope interceptor loaded appends one hop on every `send()`. A consume-then-produce service calls `IsotopeContext.adoptFromRecord(record)` between consume and produce so the trace ID and origin survive the hop.

1. How the producer interceptor gets invoked

Application code never calls `onSend` / `onAcknowledgement` directly — the Kafka client invokes them by reflection once two things are in place:

1. **Register the class in producer config.** Put `IsotopeProducerInterceptor.class.getName()` under `interceptor.classes` ([App.java:199](#), [App.java:284](#), [IsotopeTestHarness.java:96](#)). The `KafkaProducer` constructor instantiates the interceptor and owns its lifecycle.
2. **Call `producer.send(...)` as normal.** Every `send()` triggers `onSend` (caller thread, before serialization) and later `onAcknowledgement` (producer I/O thread, after broker ack or failure).

The exact call sites in `kafka-clients` 4.2.0: `KafkaProducer.send` invokes `interceptors.onSend` ([line 950](#)) and `AppendCallbacks.onCompletion` invokes `interceptors.onAcknowledgement` ([line 1600](#)). Exceptions thrown by the interceptor are caught and logged by the client's fan-out wrapper — they never break the `send` call.

2. Kafka client interceptor vs explicit library calls

This project uses one Kafka client interceptor, and it's a `ProducerInterceptor` — not a `ConsumerInterceptor`. A `ConsumerInterceptor.onConsume` sees a whole batch from `poll()`, but the application processes records one at a time, so a thread-local snapshot from the batch would be ambiguous about which record is being handled. An earlier `IsotopeConsumerInterceptor` was tried and removed for exactly that reason (see [#30](#)). The consume side instead runs on **explicit per-record library calls**: services call `IsotopeContext.adoptFromRecord(record)` between consume and produce to carry the trace through a hop (the next `send()` then sees that thread-local and appends a new hop), and `IsotopeContext.recordConsume(record, service, markerProducer)` to emit a consume-edge marker (see the next section). So: **producer interceptor for produce-side stamping; explicit per-record calls for everything consume-side.**

3. Why explicit calls for consume-side markers?

Consume-side markers — the other half of the bipartite graph. The produce-side `hops[]` chain captures `producer → topic` edges and implicitly captures consume edges for *intermediate* services (svc-B consuming from topic-AB is implied by svc-B's next produced hop), but **terminal consumers** —

services that consume but never produce — would otherwise be invisible. The bipartite story closes that gap with one library call: consumers call `IsotopeContext.recordConsume(record, service, markerProducer)` between consume and process, which forwards the inbound record's seven scalar `x-isotope-*` headers to a value-less marker record on `isotope_consume_edge_markers` and adds one new header — `x-isotope-consumer-service` — that names the consumer. The `bipartite_topology` Flink report unions these markers with the produce-side view to emit edges in both directions: `producer → topic` (from `isotope`) and `topic → consumer` (from `consume_events`). Markers are fire-and-forget: a dropped marker leaves a hole in the topology graph but never disrupts the consume/produce pipeline.

4. Why "bipartite"?

Background — why "bipartite"? The resulting topology report is an example of a **bipartite graph** from graph theory (a sub-field of discrete mathematics). A graph is *bipartite* when its vertices partition into two disjoint sets such that every edge connects a vertex in one set to a vertex in the other — edges never run within a set. Here the two sets are **services** (`svc-A`, `svc-B`, ...) and **topics** (`orders.placed`, `orders.enriched`, `orders.fulfilled`, `isotope_consume_edge_markers`); every edge is either a **produce edge** (`service → topic`, from the isotope's `hops[]`) or a **consume edge** (`topic → service`, from the `isotope_consume_edge_markers` markers). Kafka's pub/sub model guarantees the bipartite shape: services never connect directly to other services, and topics never connect directly to other topics — every interaction flows through the opposite set. Before consume-side markers existed, only produce edges were captured, so the topology view collapsed into a **unipartite** `service → service` chain with topics hidden as edge labels and terminal consumers omitted entirely. With both edge directions now wired, topics become first-class nodes alongside services, and the `bipartite_topology` report renders the full graph — every produce *and* consume edge, in both directions.